Republic of Tunisia
The Ministry of Higher Education and Scientific Research

The University of Carthage

The Higher Institute of Information and Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the

Bachelor Degree in Computer Science

Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE

Publicly defended on June 6, 2026 in front of the jury members:

Chairman:	Name SURNAME, University Relations Leader, IBM
Reporter:	Name SURNAME, Teacher, ISTIC
Examiner:	Name SURNAME, Teacher, ISTIC
Professional Supervisor:	Firas ABBESSI, Engineer, ITXTREE
Academic Supervisor:	Wassim ABBESSI, Teacher, ISTIC

Academic Year: 2025-2026

Republic of Tunisia
The Ministry of Higher Education
and Scientific Research

The University of Carthage

The Higher Institute of Information and
Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the

Bachelor Degree in Computer Science

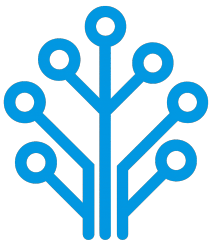
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE



Authorization of graduation project report submission:

Professional Supervisor:	Academic Supervisor:
Issued on :	Issued on :
Signature:	Signature:

Dedications

I dedicate this End-of-Study project

To my dear parents

Who have supported and encouraged me every step of the way. This is for you, in honor of all the sacrifices you have made for me. I hope to have made you both proud and happy.

To my dear sisters and brothers

Who have been a great support and a source of joy in my life. I am grateful for your love and encouragement throughout this journey.

To all my family and friends

Thank you all for your support and encouragement throughout all this time. I am truly blessed to have you in my life.

Amira BOUAFIF

Dedications

I dedicate this End-of-Study project

To my dear parents

To my dear sisters

To all my family and friends

Hajer JEMAA

Acknowledgments

We would like to extend our sincere gratitude to all those who have supported us throughout the course of this End-of-Study project. Their guidance, encouragement, and support have been invaluable throughout this journey.

First and foremost, we would like to thank our academic supervisor, Professor Wassim ABBESSI, for his guidance and insights, which have been a great help in the successful completion of this project.

Our gratitude also go to ITXTREE and, in particular, our professional supervisor, Mr. Firas ABBESSI, for providing us with the opportunity to work on this project and for their professional mentorship.

We also extend our deep appreciation to the professors at the Higher Institute of Information and Communication Technologies for their invaluable support and teachings, which have greatly contributed to our academic journey.

Finally, we thank everyone who has, in one way or another, helped in making this project a success.

Contents

List of Figures

List of Tables

General Introduction

Chapter 1

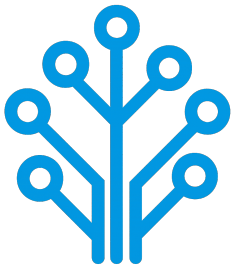
Project Context

1.1 Introduction

This chapter serves as an introduction to the project. We begin with a presentation of the company hosting the internship, a study of existing network intrusion detection systems. Then, we present the problem and the proposed solution. Finally, we end the chapter with a conclusion.

1.2 Presentation of the company

ITXTREE is an early-stage technology startup based in Sidi Rzig, Tunisia, focused on building a solid foundation for future digital solutions it is built on the core values of integrity, quality, curiosity and adaptability.



1.3 Study of the existing

Cybersecurity is a highly sensitive domain where the ability to react quickly can determine the impact of an attack. The earlier a threat is identified, the more effectively its consequences can be limited.

For this reason, Network Intrusion Detection Systems (NIDS) play a central role in monitoring and analyzing network traffic. The most widely used approach relies on:

- **Signature-based:** These systems analyze network traffic, often through deep packet inspection and compares it against a signature database. When a match is found an alert is triggered. Well-known tools such as Snort [1] and Suricata [2] are representative of this approach.

- **Anomaly-based:** Anomaly detection is learning normal behavior of the network through statistical analysis of traffic parameters. If there is a sudden increase of traffic on a port outside of normal thresholds, it might be an indication of a brand-new threat [1].

1.4 Criticism of the existing

Despite their widespread use, signature-based NIDS present several limitations:

- **High Maintenance and Reactivity:** Their effectiveness is essentially tied to the completeness and accuracy of their signature rule sets. As attack techniques continuously evolve, these systems require frequent updates to remain effective. This maintenance process is both time-consuming and reactive since new signatures can only be created after an attack has already been identified and analyzed.

- **Performance:** As the number of rules increases, the system must process a larger set of comparisons for each packet, which can lead to slower detection. In a context where reaction time is critical, this degradation directly affects the system's ability to respond effectively.

- **Rigidity and Evasion Vulnerability:** Attackers can exploit the rigidity of signature-based detection systems by introducing slight variations in attack patterns, allowing them to evade existing signatures while preserving the same malicious intent. Since many signature rules are publicly available or widely shared with the security community, attackers may analyze them to identify gaps and design variants that bypass detection mechanisms.

Overall, these systems rely on exact matching and predefined logics which limits their adaptability in the face of evolving and increasingly sophisticated threats.

1.5 Proposed solution

With the growing integration of Artificial Intelligence (AI) in security systems, new approaches aim to overcome the limits of traditional methods. To address these challenges, we propose an AI-driven NIDS capable of learning patterns from network traffic and classifying different types of network activity instead of relying solely on predefined rules.

1.6 Project Pipeline

A project pipeline is a structured sequence of stages that outlines the workflow and processes involved in the development and deployment of a project. It serves as a roadmap, guiding the team through various phases, from initial planning to final delivery.

Figure 1.1 illustrates the project pipeline for our network intrusion detection system.

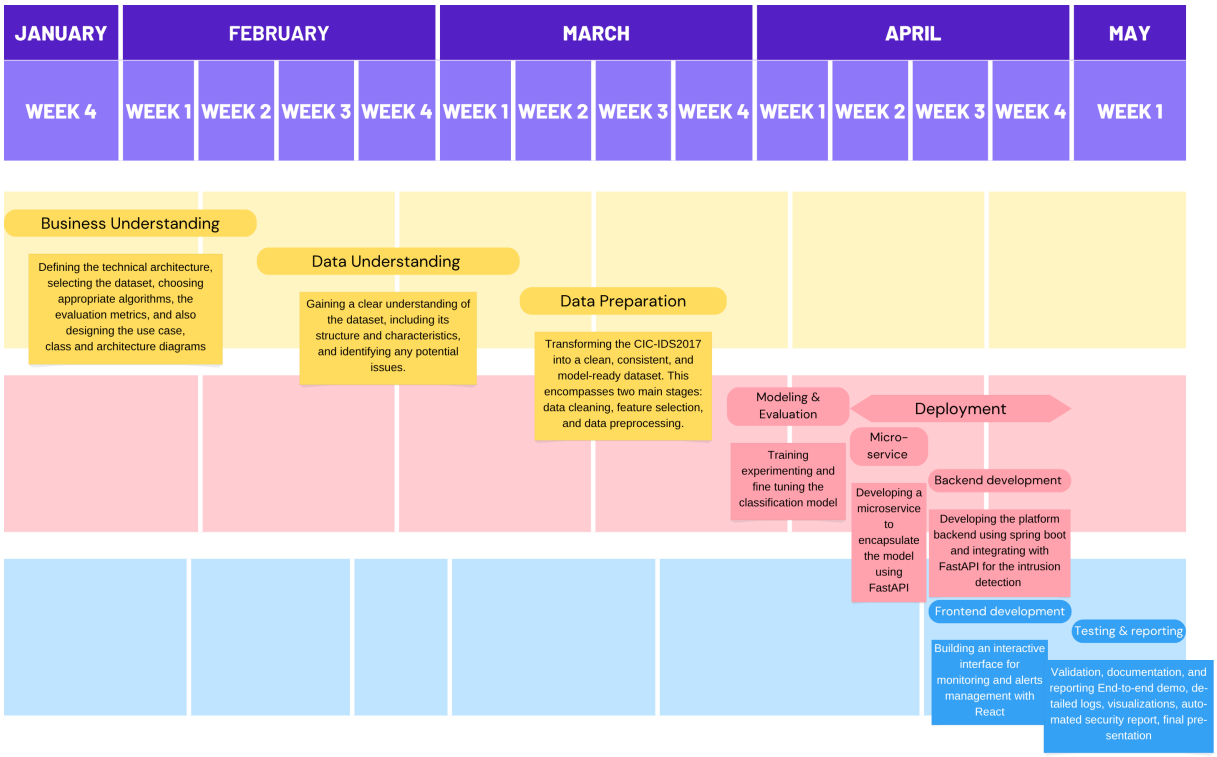


FIGURE 1.1 – Project Pipeline

1.7 Methodologies

In this section, we present the methodologies that we use in this project, including the machine learning methodology CRISP-DM and the software development methodology UML.

1.7.1 CRISP-DM

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a widely used framework for structuring machine learning projects. It consists of six phases:

- **Business Understanding:** Defining project objectives and requirements and gaining a clear understanding of the business problem.

- **Data Understanding:** Collecting and exploring the data to gain insights and assess its quality by identifying data quality issues.
- **Data Preparation:** Cleaning and preparing the data for modeling by selecting relevant features, handling missing values and splitting the dataset.
- **Modeling:** Selecting and applying appropriate modeling techniques, building, testing and tuning models to optimize their performance.
- **Evaluation:** Evaluating the model's performance and determining if it meets the business objectives.
- **Deployment:** Deploying the model into production, monitoring and maintaining the model's performance.

Figure 1.2 illustrates these phases and their relationships.

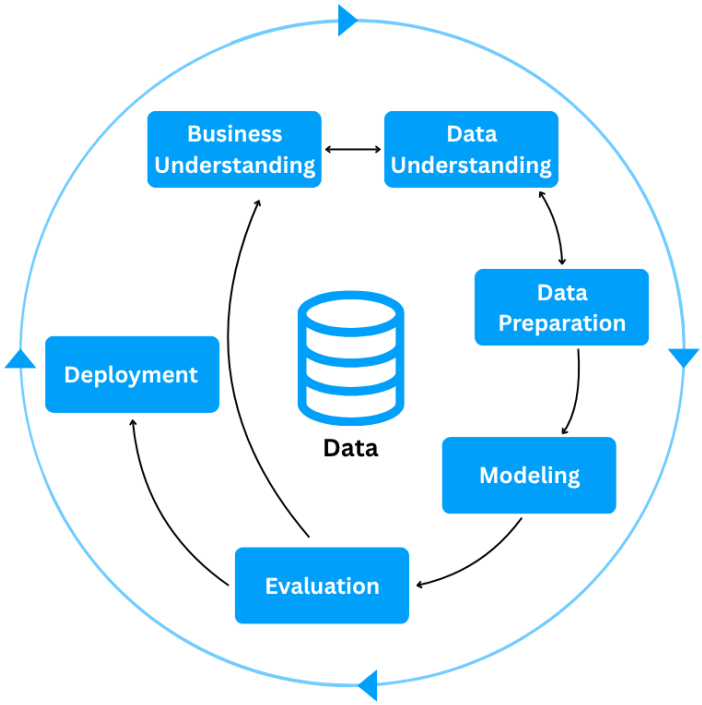


FIGURE 1.2 – CRISP-DM Methodology

1.7.2 UML

For the modeling of the software system, we use UML (Unified Modeling Language), a standardized modeling language used to specify, visualize, construct, and document the artifacts of software systems through diagrams such as use case, class, and sequence diagrams. There are two types of UML diagrams:

- **Behavioral Diagrams:** State machine, activity, use case, sequence, communica-

tion and interaction overview diagrams.

- **Structural Diagrams:** Class, object, component, deployment and package diagrams.

UML makes complex systems easier to design and communicate, it provides a common language for developers, architects, and stakeholders [3].

1.8 Conclusion

In this chapter, we introduce the host company ITXTREE, study existing network intrusion detection systems, discuss their shortcomings and finally present a solution. In the next chapter we specify the system requirements.

Chapter 2

Requirements Specification

2.1 Introduction

Throughout this chapter, we specify both the functional and non-functional requirements, identify the system actors, and present the use case diagram. By defining these elements, we aim to establish a clear and shared understanding of the system's expected behavior and constraints, providing a solid foundation for the subsequent design and implementation phases.

2.2 Functional Requirements

Functional requirements define *what* the system should perform. The main functionalities of our system are as follows:

— Authenticate;

— View Dashboard;

— View Statistics summary;

— Trigger Network Analysis;

— View Alerts List;

— View Reports;

2.3 Non-Functional Requirements

Non-functional requirements define *how* the system should perform its functions. They describe the quality attributes and constraints of the system:

- **Performance:** The system must process network traffic and generate analysis results with minimal latency, ensuring fast detection of potential intrusions.
- **Security:** The system must ensure secure authentication and enforce role-based access control, preventing unauthorized actions and protecting system integrity, while supporting non-repudiation to guarantee accountability and traceability of all operations.
- **Maintainability:** The system should be easy to maintain, allowing for efficient debugging, updates, and modifications to existing components.
- **Usability:** The system should provide an intuitive interface, enabling users to efficiently navigate and interact with its functionalities.

2.4 Actors Identification

Actors are external entities, users, organizations, or external systems that interact with the system to achieve one or more functional requirements, with each actor representing a specific role.

In the context of our project, we identify two primary actors, as illustrated in Figure 2.1

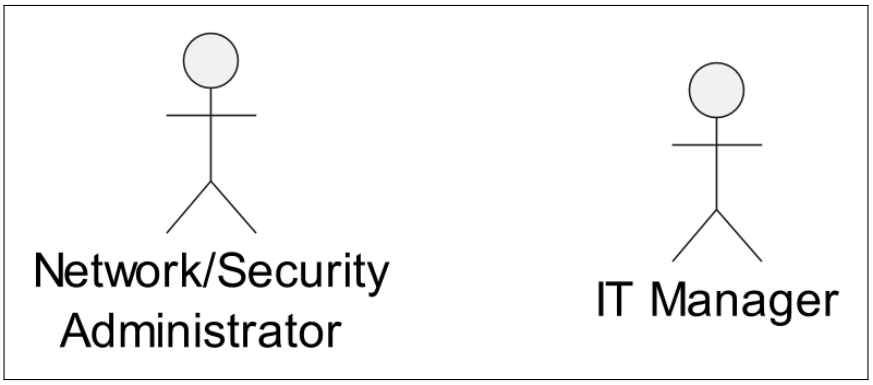


Figure 2.1 – Actors

Table 2.1 presents the roles of each actor in our system.

Table 2.1 – Roles of Actors

Actor	**Role**
Network/Security Administrator	- Authenticate - View Dashboard - View Statistics Summary - Trigger Network Analysis - View Alerts List
It Manager	- Authenticate - View Dashboard - View Statistics Summary - View Reports

2.5 Use Case Diagram

A use case diagram is a visual representation of *how* users interact with the system and *what* a system does through different use cases. Figure 2.2 illustrates that diagram.

2.6 Conclusion

In this chapter, we outline the functional and non-functional requirements of our system, identify the primary actors, and represent the way they interact with the system and the system's functionality via a use case diagram. In the next chapter, we present the conceptual design of the platform.

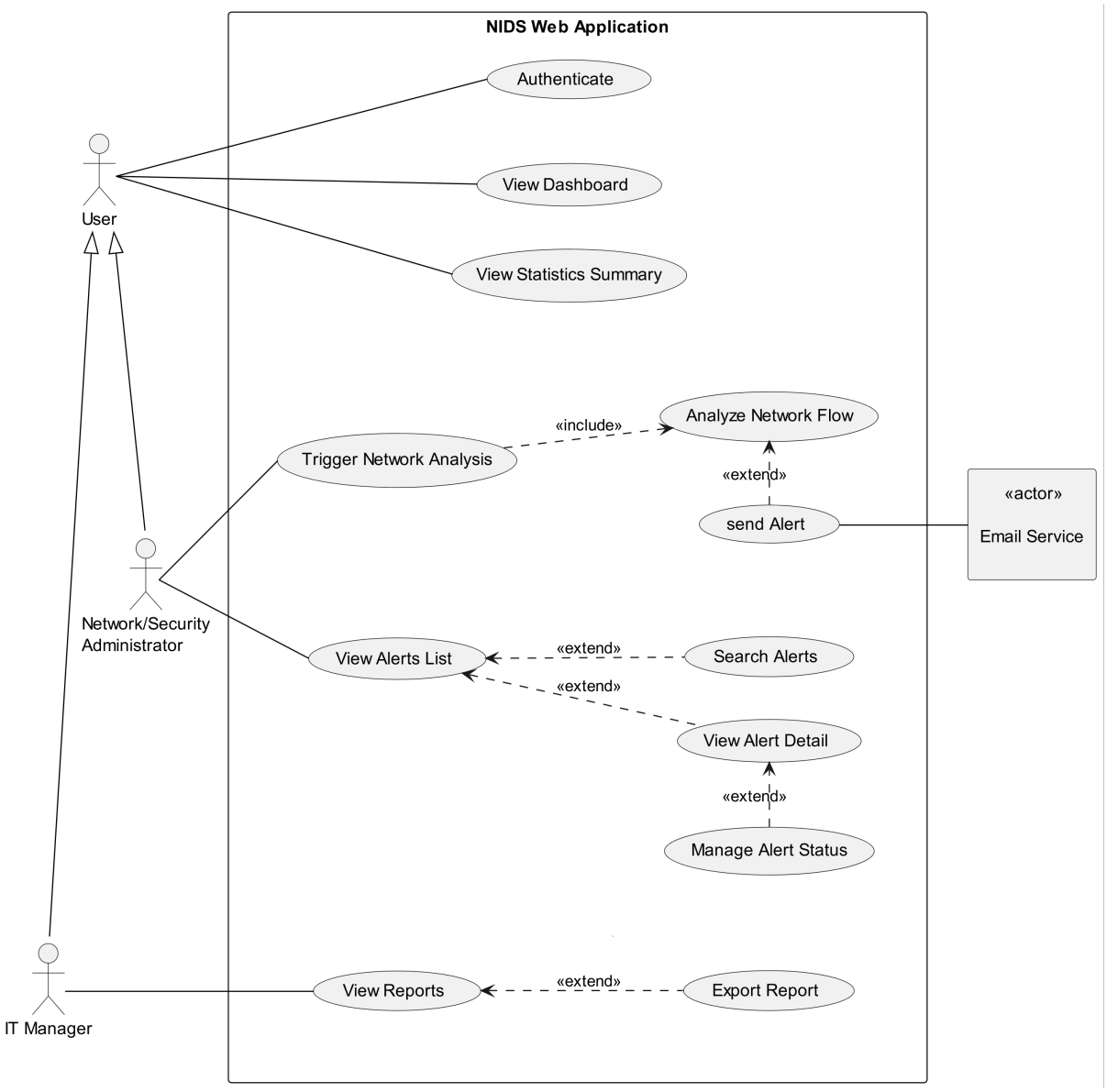


FIGURE 2.2 – Use Case Diagram

Chapter 3

Conceptual Design

3.1 Introduction

3.2 Sequence Diagrams

3.3 Class Diagram

3.4 Deployment Diagram

3.5 Conclusion

Chapter 4

Data Collection and Preparation

4.1 Introduction

Data constitutes the foundation of any AI system; the quality of all subsequent stages, including model training and performance, is directly dependent on it. For this reason, it requires particular attention.

This chapter presents the complete data workflow adopted in this project. It begins with data collection, followed by an in-depth exploration phase aimed at understanding the structure, characteristics, and underlying patterns of the dataset. Based on the insights obtained during this stage, we then proceed to data preprocessing, where appropriate transformations and refinements are applied to ensure the dataset is properly structured and suitable for model training.

4.2 Data Collection

This section introduces the process of acquiring the data used in this project. In general, this phase may involve either constructing a dataset from scratch or relying on existing sources. In our case, the decision to use a public dataset is mainly dictated by the nature of the cybersecurity domain.

Unlike other fields, network intrusion data cannot be collected on demand, as it depends on the occurrence of unpredictable and potentially harmful attacks. In addition, data collection in real operational environments is often constrained due to privacy concerns, security risks, and limited access to organizational traffic. Furthermore, even when such data is available, obtaining reliable ground truth labels is difficult and typically requires extensive manual analysis.

For these reasons, constructing a proprietary dataset is impractical, particularly in the absence of real network traffic provided by the host organization. Consequently, the use of a public dataset becomes a necessary and coherent choice, as such datasets are generated in controlled environments and include labeled instances based on expert-defined attack scenarios.

4.2.1 Dataset comparative

We narrowed it down to these three datasets. Below is a comparative table 4.1 using our primary selection criteria.

TABLE 4.1 – Comparison of network intrusion detection datasets

Dataset	**Source**	**Size**	**Data Generation Approach**	**Attack Diversity and Temporal Relevance**	**Usability**
NSL-KDD [4]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Small (≈148k records)	Derived from the KDD '99 dataset, its traffic was generated in a military-style network environment with predefined automated attack scripts.	Low; 4 attack families: DoS, Probe, R2L, U2R with several specific attacks in each family.	High; legacy benchmark widely used in academic studies, mainly for baseline comparison and educational purposes.
UNSW-NB15 [2]	Australian Centre for Cyber Security (ACCS), UNSW Canberra	Medium (≈2.5M records)	Synthetic traffic generated with IXIA PerfectStorm tool.	Medium; 9 attack types: Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms.	High; widely used modern benchmark for machine learning-based intrusion detection research.
CIC-IDS-2017 [3]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Medium (≈2.8M records)	Traffic generated in an enterprise-like network environment composed of attacker and victim machines, using real attack tools.	High; 14 attack types: Includes the most common attacks based on the 2016 McAfee report, such as Web based, Brute force, DoS, DDoS, Infiltration, Heart-bleed, Bot and Scan.	Very High; one of the most widely adopted and current standard datasets in intrusion detection research.

The comparative analysis of the three datasets reveal that each has its own limitations. NSL-KDD, although still relevant, exhibits limitations in size and temporal relevance, as it is relatively old and does not reflect recent network behavior and modern attacks types. UNSW-NB15 is larger and offers a more recent alternative, but it remains limited in terms

of attack type coverage, and its traffic is fully synthetically generated. This makes CIC-IDS2017 the most suitable dataset, as it provides a more realistic experimental setting, a wider range of the most common attack types, and it is also widely adopted in recent research studies.

4.2.2 Dataset overview

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB), within a controlled environment, where the network traffic was captured in packet capture (PCAP) format.

The transformation of raw PCAP files was done using a feature extraction tool, CICFlowMeter. Therefore a row in this dataset doesn't represent a packet, it represents an overview of the session behaviour between two endpoints through statistical summary.

4.3 Data Understanding

4.3.1 Data quality

The quality of a model's output is directly tied to its input. In this section we evaluate the dataset for any quality issues that could disrupt model training such as missing values (NaN - Not a number), infinite values (inf), negative values, duplicates rows, constant and near-constant features. For each issue, the analysis investigates the root cause, assesses its global impact on the dataset, across classes and whether it endangers any minority classes.

NaN and Infinite Values: Scanning the dataset for undefined values revealed that NaN values appear exclusively in `Flow Bytes/s`, while infinite values appear in both `Flow Bytes/s` and `Flow Packets/s`. To understand the origin, the CICFlowMeter source code was inspected directly. The relevant formulas are:

$$\text{Flow Bytes/s} = \frac{\text{forwardBytes} + \text{backwardBytes}}{\text{flowDuration}/1{,}000{,}000}$$

$$\text{Flow Packets/s} = \frac{\text{packetCount (fwd + bwd)}}{\text{flowDuration}/1{,}000{,}000}$$

Both features divide by `Flow Duration`. When `Flow Duration = 0`, any nonzero numerator yields Inf, and a zero numerator yields NaN 0/0. This distinction maps directly onto two observable row types in the dataset:

- Rows with 2 forward packets, 0 backward, and nonzero byte count: `Flow Duration` = 0 with bytes > 0, producing `Flow Bytes/s` = Inf.
- Rows with 1 forward packet, 1 backward packet, and zero byte count: `Flow Duration` = 0 with zero bytes, producing `Flow Packets/s` = NaN.

The zero byte count in the second case, despite existing packets, is because CICFlowMeter computes `Total Length of Fwd/Bwd Packets` from the packet payload only. Packets carrying no data content only TCP headers (ACK, SYN) produce zero payload length. The true anomaly in both cases is `Flow Duration = 0`.

`Flow Duration` is computed as the timestamp difference between the last and first packet in a flow. It evaluates to zero when two packets arrive within the same timestamp resolution window as the tool lacks the granularity to distinguish arrivals that are nanoseconds apart. Every affected row contains exactly two packets total, either two forward or one forward and one backward, which is consistent with this explanation: with only two packets, a single resolution failure collapses the entire duration to zero. This collapse propagates to every other feature that depends on the timestamp value, like `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, and all rate-based features reduce to zero as well.

These rows impact is as follows. Globally, 353 unique row carry at least one NaN value making it 0.01% of the dataset, and 1,564 unique row carry at least one Inf value making it 0.05% of the dataset. At class level, Inf impact BENIGN with 1,427 unique row making it 0.06% of the class samples and only four attack classes are affected. NaN infects only BENIGN with 350 making it less then 0.01% of the class samples along with Dos Hulk. Rare classes such as Heartbleed, Infiltration, and Web Attack – SQL Injection are not affected. The attack class-level distribution is shown in the 4.1 Figure.

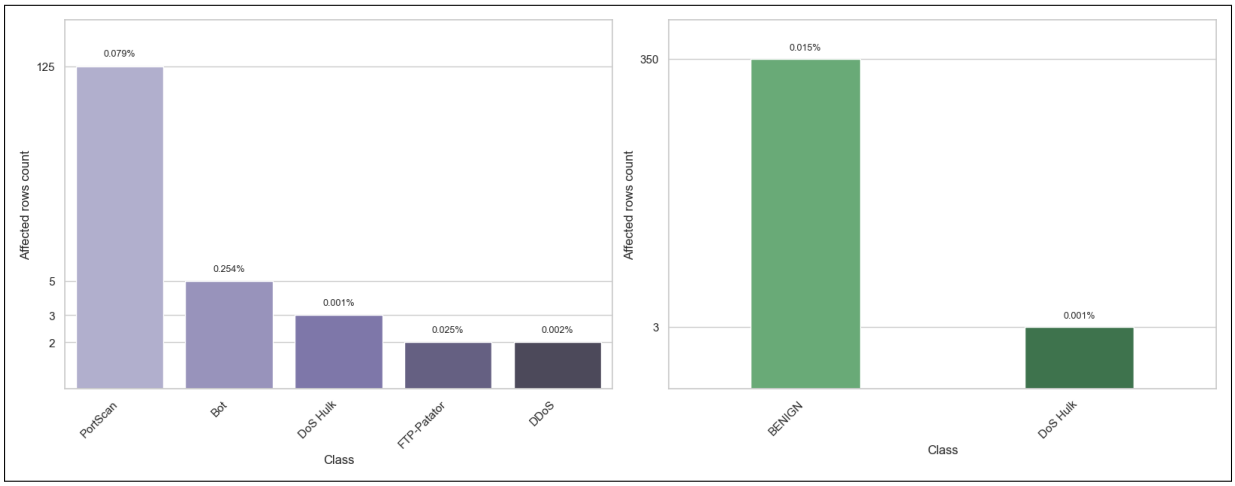


FIGURE 4.1 – Inf and NaN attack class distribution

Negative Values: Given the nature of the dataset feature, their logical value should never be negative yet Negative values were found across several feature groups, each with a distinct cause.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

  These two features carry negative values at scale: 1,001,189 rows (35.4%) and 1,441,552 rows (50.9%) respectively hold the value -1. This is not a measurement error. CICFlowMeter uses -1 as a sentinel to signal that the TCP initial window size could not be captured, either because the SYN packet was absent from the capture, or because the flow is UDP-based and has no TCP handshake. It is a deliberate

encoding choice by the tool, not a data corruption.

- **Packet ordering artifacts:** `Flow Duration`, `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, `Fwd IAT Min`, `Flow Bytes/s`, and `Flow Packets/s`.

  A total of 115 rows carry negative values in `Flow Duration` ranging from -13 to -1, with the same negative values propagating identically into `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min`.

  This is a cascade, since each of these rows has exactly one packet forward and one backward, there is only one inter-arrival interval in the flow, so mean, max, and min all equal `Flow Duration` exactly.

  Division by a small negative duration scaled by 1,000,000 then causes `Flow Bytes/s` and `Flow Packets/s` to carry large negative values.
  For example, with `Flow Duration` $= -1$ and 12 bytes of payload:

$$\texttt{Flow Bytes/s} = \frac{12}{-1/1{,}000{,}000} = -12{,}000{,}000$$

  This is likely to be a packet ordering artifact when processing the packets; additionally, CICFlowMeter performs no guard against negative duration, so the error propagates unchecked. The affected rows are exclusively BENIGN traffic and represent less than 0.0001% of the dataset.

  After excluding these 115 rows, an additional 2,776 rows exhibit negative `Flow IAT Min` values and 17 rows exhibit a negative `Fwd IAT Min`. Here the ordering artifact affects only a single packet pair within a longer multi-packet flow, dragging the minimum IAT negative while all other temporal features remain valid. The attack class-level distribution is shown in the 4.2 Figure.

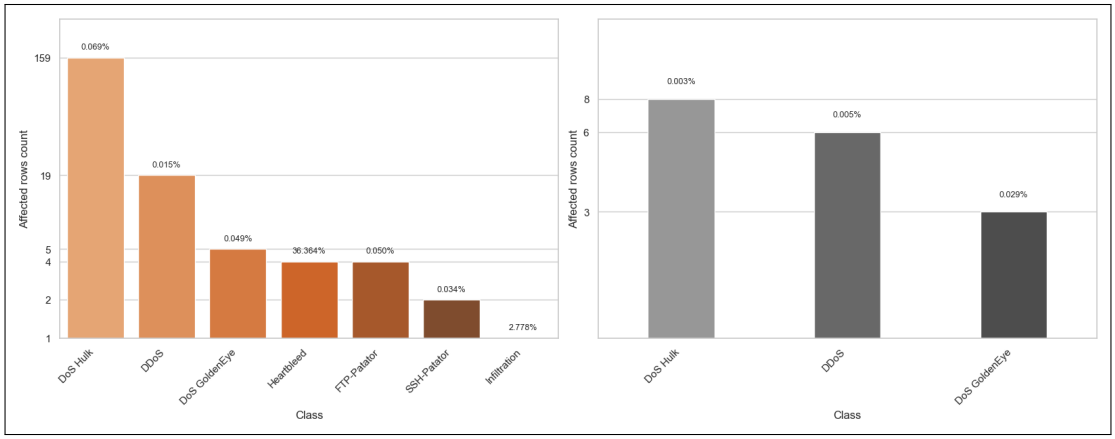


FIGURE 4.2 – Ordering Artifact Class Distribution

- **UDP parsing bug:** `Fwd Header Length`, `Fwd Header Length.1`, `Bwd Header Length`, `min_seg_size_forward`.

  These features exhibit values in the range of -32,212,234,632 to -1,073,741,320, far outside any legitimate segment size and a TCP header size, which is bounded between 20 and 60 bytes.

The root cause is a bug in CICFlowMeter: when processing UDP packets, the tool reads the header size from a TCP-specific field that simply does not apply to UDP, producing meaningless values that corrupt these features.

The bug is confirmed by the data: all 35 affected rows are UDP flows, as indicated by `Init_Win_bytes_forward` = `Init_Win_bytes_backward` = -1.

The impact is narrow, 35 rows in `Fwd Header Length`, `Fwd Header Length.1`, and `min_seg_size_forward`, and 22 rows in `Bwd Header Length`, all exclusively BENIGN traffic, collectively under 0.0001% of the dataset.

Exact duplicates: Duplicate rows are common in network datasets since a row in the dataset does not represent a packet but a session behaviour summarised through statistical aggregation. Many flows share identical statistical profiles, thus the presence of duplicates does not necessarily indicate a data error.

Globally, duplicate rows represent 308,381 out of 2,830,743 rows (10.89%). Their distribution by attack class is as show in the 4.3 figure and no minority class is affected.

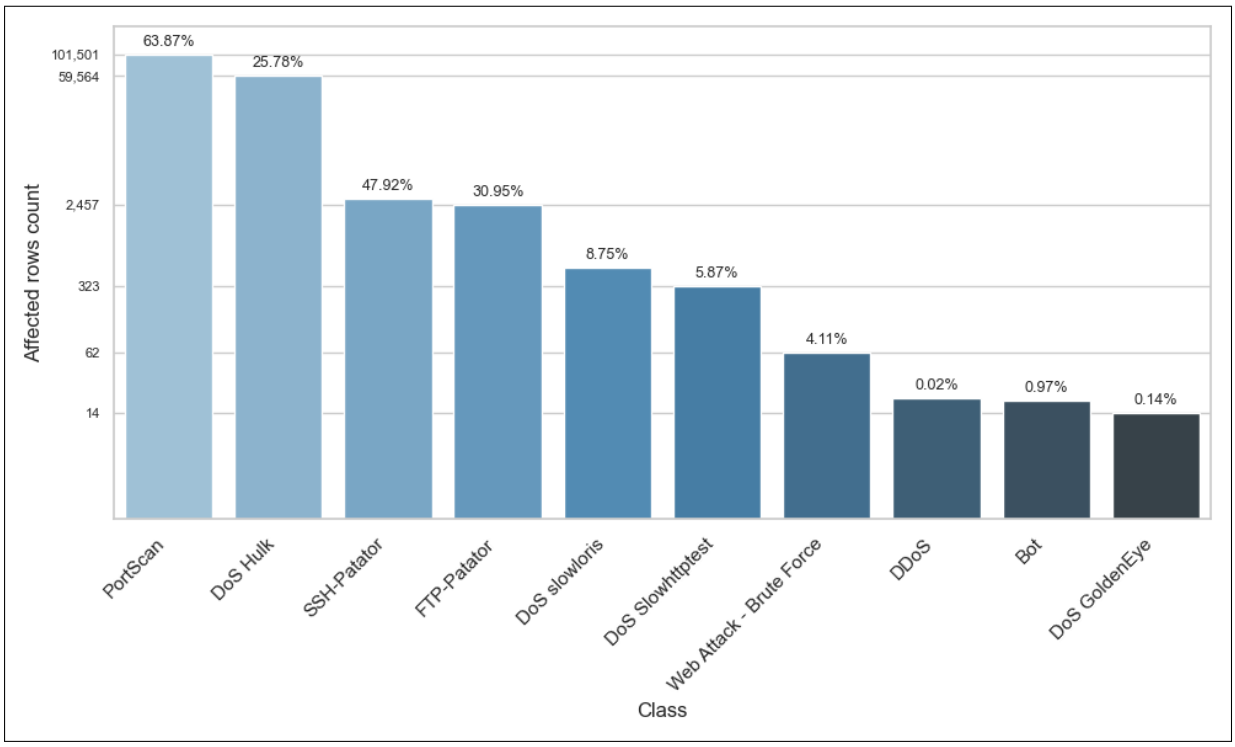


FIGURE 4.3 – Exact duplicates attack class distribution

Contradictory duplicates: These rows share the exact same feature values except for the target column, their labels differ. CICFlowMeter aggregates raw packet-level captures into flow statistics. Two flows from different sessions, one BENIGN and one malicious, can produce statistically identical feature vectors if their traffic patterns are indistinguishable at the flow-aggregation level. This is a fundamental limitation of statistical flow-based features rather than a labelling error.

Globally, 7,020 rows are affected. BENIGN traffic is the dominant participant in

contradictory groups, as expected, BENIGN flows vastly outnumber attack flows, so the probability of a BENIGN flow sharing a feature vector with an attack flow is higher than inter-attack collisions. Contradictions involve only well-represented classes: DoS Hulk, DDoS, DoS slowloris, and PortScan. The 4.4 figure illustrates their distribution.

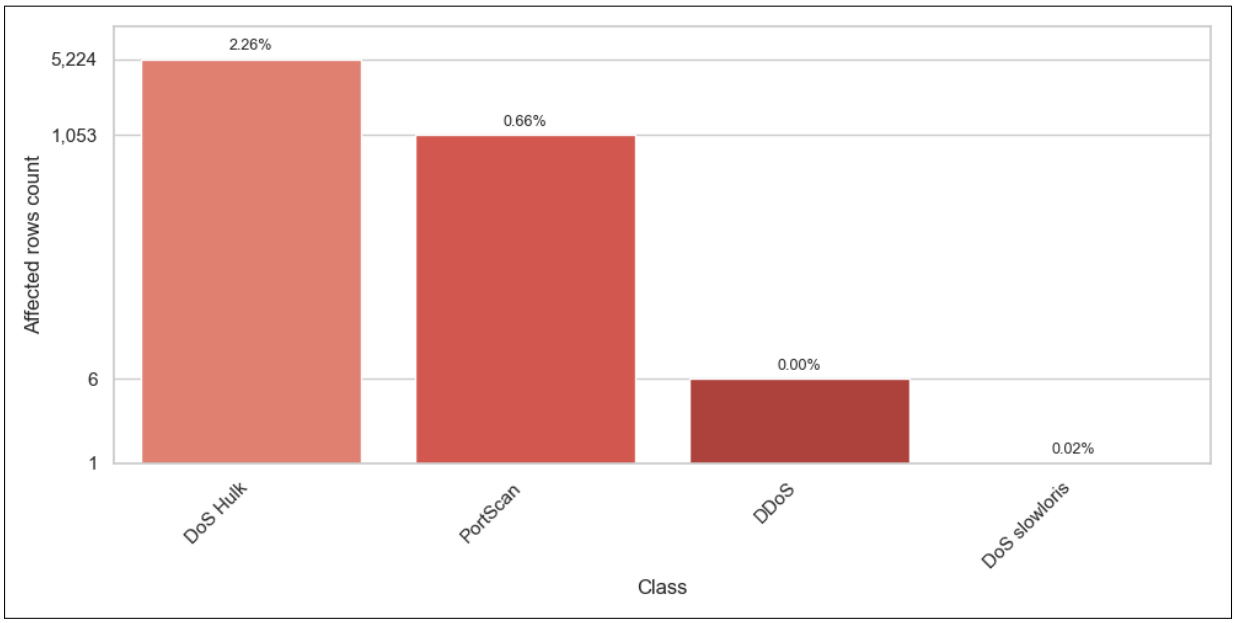


FIGURE 4.4 – ontradictory duplicates attack class distribution

Constant features: Constant features are those with a single unique value across the entire dataset, in this case, always zero. To understand this behaviour, each feature was traced back to its computation logic in the CICFlowMeter source code. The root cause divides them into two groups.

- **Bulk transfer features:** `Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`.

  Bulk transfer refers to sending a large number of packets consecutively in one direction, with very little time between them.

  These features are derived from CICFlowMeter's bulk detection logic, which only activates when at least four consecutive packets carrying actual data are observed in close succession.

  If that condition is never met, either because flows in this dataset rarely involve this kind of continuous data sending, or because many packets carry no payload, the counters stay at zero and all six features are exported as zeros.

  This is exactly what is observed. These features are constant not by error, but because the bulk detection logic simply never triggers on this dataset.

- **Backward flag counters:** `Bwd PSH Flags` and `Bwd URG Flags`

  These are constant zero due to a bug in CICFlowMeter's implementation. The features `Bwd PSH Flags` and `Bwd URG Flags` are meant to count how many times

the PSH and URG flags appear in packets traveling in the backward direction.

However, these counters are only incremented inside the method that processes the first packet. Since the first packet by definition can only be forward, CICFlowMeter assigns the source IP from that first packet and compares subsequent packets against it to determine direction. The method that handles the following packets never updates these counters, so both features remain zero for every flow, regardless of the actual traffic.

Near-constant features: `Fwd URG Flags`, `CWE Flag Count`, `RST Flag Count`, and `ECE Flag Count`.

Each feature hold exactly two unique values across the entire dataset. These are rare-event TCP control flags that are virtually absent in normal traffic. Combined, their non-zero occurrences account for only 315 rows (0.0139%), all within BENIGN traffic. Therfore, they carry no discriminative signal for any attack class.

4.4 Data preparatation

This section translates the findings from the data understanding phase into concrete preprocessing actions.

4.4.1 Data cleaning

The integrity of data is a prerequisite for any reliable model. Thus, we leverage data correctness over sample retention, as long as data volume and class balance are not meaningfully affected.

NaN and Inf Values: Rows carrying NaN or Inf values in `Flow Bytes/s` and `Flow Packets/s` are removed. As established, these arise from division by a zero `Flow Duration`, due to the lack of time granularity. Beyond the invalid rate features, these rows are also dominated by zeros which do not reflect the true nature of the flow, but are simply an artifact of insufficient measurement precision. Such a row cannot serve as a valid representation of a network session, as its defining characteristics were lost in the granularity gap.

The scale reinforces the decision. Globally, fewer than 0.11% of rows are affected. Additionally, the unique affected rows per attack class amount to 5 for Bot, 125 for PortScan, 2 for DDoS, and 3 for DoS Hulk, representing less than 0.00% of any single class, all of which carry sufficient clean samples to generalise from. No minority class is affected.

Negative Values: Negative values across the dataset fall into four distinct categories, each handled differently.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

  These are retained without modification. As noted, -1 is a deliberate sentinel used by CICFlowMeter to encode the absence of a TCP handshake, it is semantically valid and carries information about the flow type. Replacing it would discard a meaningful signal.

- **Packet ordering artifacts:** This anomaly originates during packet processing, since these sessions consist of a single packet exchange, the flow has no other packets to establish a valid duration or inter-arrival times. `Flow Duration` was rendered negative, and all rate-based features, `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min` collapsed to that same negative value, leaving no part of the flow intact.

  Beyond the specific features where the corruption is visible, there is a deeper concern. This is a bug introduced during dataset construction, and its effects are not fully observable from the CSV alone.

  Negative values in `Flow Duration` make the anomaly detectable, but there is no guarantee that the ordering artifact confined its damage only to features that produce obviously invalid numbers.

  The same error may have silently distorted other features into values that appear plausible but are incorrect, damage that is invisible precisely because it does not manifest as negatives or zeros, but as ordinary-looking numbers that simply do not reflect the true flow.

  In order to preserve data integrity and given the rows effect is negligible, only 115 rows belonging to BENIGN traffic, we decide to drop these rows.

  `Flow IAT Min` and `Fwd IAT Min` share the same root cause, a packet ordering artifact, but the outcome is fundamentally different. Here the affected flows span multiple packets, so the ordering error disturbed only a single packet pair within an otherwise well-measured session.

  The negative value appears only in the minimum IAT statistic, while `Flow Duration` remains positive and all other features are valid.

  Inter-arrival time cannot be negative by definition, so clipping both features to zero restores the value to the physical boundary it should never have crossed, without fabricating anything.

  This targeted correction is justified precisely because the flow itself was captured correctly, unlike the single-packet cases above, where the corruption was total and removal was the only option.

  This treatment also preserves the 4 Heartbleed rows that would otherwise be discarded, a meaningful consideration given the class contains only 11 samples in total.

- **UDP parsing bug:** `Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`, and `min_seg_size_forward`.

  These rows are removed. As established, they are caused by a parsing bug rendering these feature values meaningless and irrecoverable. The fact that all affected rows belong exclusively to BENIGN traffic and represent under 0.002% of the dataset

confirms that removal incurs no meaningful cost to class balance or dataset integrity.

Exact duplicates: Exact duplicate rows are dropped. Retaining them risks data leakage, a row could appear in both the training and test sets simultaneously, moreover it risks overfitting by biasing the model toward a single flow pattern. Since no minority classes are affected, the removal carries no risk to rare attack samples.

Contradictory duplicates: These rows present a serious risk to model training. An identical feature vector mapped to more than one target is unresolvable, retaining such rows forces the model to fit contradictory labels from the same input. All rows belonging to contradictory groups are therefore dropped entirely, regardless of their label. Since rare classes are unaffected, this carries no risk of losing minority attack samples.

4.4.2 Feature engineering

4.4.3 Label encoding

The selected model, XGBoost, requires numerical input for both features and the target variable. While all features are already numerical, the target column `Label` is categorical and stored as an object type. Hence, Label Encoding was performed using Label encoder from sklearn.preprocessing. The encoder was fitted and applied to the target column in a single step using `fit_transform()`. To preserve interpretability of the model's predicted outputs, the resulting integer-to-class mapping was exported as a `label_mapping.json` file derived from `le.classes_`, allowing any predicted label to be traced back to its original class name.

Train/ Test split

Several factors make us approach this step with caution, when assessing the train/test split strategies.

A per-file split is not an option, as the files are divided per attack type, each attack class appears in exactly one file, so splitting by file would result in classes entirely absent from either the training or test set.

A standard random 80/20 split presents its own issues. The CIC-IDS2017 dataset is heavily imbalanced, the BENIGN to attack ratio is 80/20 Additionally, rare classes present less than 0.001

For these reasons, a stratified train/test split is selected. It ensures all classes are present in both the training and test sets, each maintaining their original proportions, so the class distribution in each split mirrors that of the full dataset.

However, this strategy doesn't solve the class imbalance. This issue will be addressed and treated more during the model training.

4.5 Conclusion

This chapter presented the full data workflow adopted for this project, from selecting an appropriate public intrusion detection dataset to preparing it for model training. We compared candidate datasets and justified the choice of CIC-IDS2017, then analyzed data quality issues such as NaN/Inf values, negative artifacts, duplicates, and constant features, linking each issue to its root cause in traffic generation or feature extraction. Finally, we applied cleaning and preparation steps that preserve class coverage and prevent leakage, and we defined an encoding and stratified splitting strategy to support reliable evaluation in the subsequent modeling chapter.

Chapter 5

Modeling

5.1 Introduction

5.2 State of the Art

5.3 Adopted Approach

5.4 Training and Hyperparameter Optimization

5.5 Conclusion

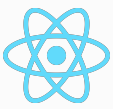
Chapter 6

Realization

6.1 Introduction

6.2 Development Tools and Technologies

TABLE 6.1 – Development tools and technologies

Language	
	**Python**
Machine Learning	
	**Scikit-learn**
Backend	
	**Spring Boot**
Frontend	
	**React**

Database

MySQL

Local Server

XAMPP

API Testing

Postman

API

FastAPI

Containerization

Docker

Orchestration

Docker Compose

Version Control

Git

Code hosting and collaboration

GitHub

IDE

VS Code

IDE

IntelliJ IDEA

6.3 Performance Analysis and Evaluation Metrics

6.4 User Interfaces

This section presents the main user interfaces of the NIDS Web Application.

6.4.1 Use case «Authenticate» realization

The authentication interface allows the user to log in and access the platform features, as shown in Figure 6.1.

Placeholder screenshot for the **Authenticate** use case.

FIGURE 6.1 – Authentication interface (placeholder)

6.4.2 Use case «View Dashboard» realization

The dashboard interface provides a global overview of the system state and recent activity, as shown in Figure 6.2.

Placeholder screenshot for the **View Dashboard** use case.

FIGURE 6.2 – Dashboard interface (placeholder)

6.4.3 Use case «View Statistics Summary» realization

The statistics summary interface provides aggregated metrics and trends for monitoring purposes, as shown in Figure 6.3.

Placeholder screenshot for the **View Statistics Summary** use case.

FIGURE 6.3 – Statistics summary interface (placeholder)

6.4.4 Use case «View Alerts List» realization

The alerts list interface displays all detected alerts with key information for quick review, as shown in Figure 6.4.

Placeholder screenshot for the **View Alerts List** use case.

FIGURE 6.4 – Alerts list interface (placeholder)

6.4.5 Use case «View Alert Detail» realization

The alert detail interface presents the full information of a selected alert, enabling deeper investigation, as shown in Figure 6.5.

Placeholder screenshot for the **View Alert Detail** use case.

FIGURE 6.5 – Alert detail interface (placeholder)

6.4.6 Use case «Search Alerts» realization

The search interface enables filtering and searching through alerts by different criteria, as shown in Figure 6.6.

6.4.7 Use case «Manage Alert Status» realization

The alert status management interface allows updating the handling state of an alert (e.g., new, in progress, resolved), as shown in Figure 6.7.

6.4.8 Use case «Trigger Network Analysis» realization

The analysis trigger interface allows an administrator to start a network analysis process, as shown in Figure 6.8.

6.4.9 Use case «Analyze Network Flow» realization

The network flow analysis interface displays analysis outputs and indicators extracted from network traffic, as shown in Figure 6.9.

Placeholder screenshot for the **Search Alerts** use case.

FIGURE 6.6 – Search alerts interface (placeholder)

Placeholder screenshot for the **Manage Alert Status** use case.

FIGURE 6.7 – Manage alert status interface (placeholder)

6.4.10 Use case «send Alert» realization

The alert sending interface triggers an alert notification to external services (e.g., email), as shown in Figure 6.10.

6.4.11 Use case «View Reports» realization

The reports interface allows the user to browse and consult generated reports, as shown in Figure 6.11.

6.4.12 Use case «Export Report» realization

The export interface enables downloading a report in a suitable format for sharing or archiving, as shown in Figure 6.12.

6.5 Integration

6.6 Conclusion

Placeholder screenshot for the **Trigger Network Analysis** use case.

FIGURE 6.8 – Trigger network analysis interface (placeholder)

Placeholder screenshot for the **Analyze Network Flow** use case.

FIGURE 6.9 – Analyze network flow interface (placeholder)

Placeholder screenshot for the **send Alert** use case.

FIGURE 6.10 – Send alert interface (placeholder)

Placeholder screenshot for the **View Reports** use case.

FIGURE 6.11 – Reports interface (placeholder)

Placeholder screenshot for the **Export Report** use case.

FIGURE 6.12 – Export report interface (placeholder)

General Conclusion

Webography

[1] https://www.snort.org/. Accessed on: April 6, 2026.

[2] https://suricata.io/. Accessed on: April 6, 2026.

[3] https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/. Accessed on: April 2, 2026.

Bibliography

[1] Chris Jackson. *Network Security Auditing*. 800 East 96th Street, Indianapolis, IN 46240 USA: Cisco Press, 2010. ISBN: 978-1-58705-352-8.

[2] Nour Moustafa and Jill Slay. "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)". In: *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015. DOI: 10.1109/MilCIS.2015.7348942.

[3] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization". In: *ICISSP* (2018), pp. 108–116. DOI: 10.5220/0006639801080116.

[4] Mahbod Tavallaee et al. "A Detailed Analysis of the KDD CUP 99 Data Set". In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.